

১. Timing Diagram (টাইমিং ডায়াগ্রাম) কি?

সহজ কথায়, টাইমিং ডায়াগ্রাম হলো একটি গ্রাফ বা ছবি যা দেখায় যে সময়ের সাথে সাথে মাইক্রোপ্রসেসরের ভেতরে কী কী ঘটছে।

যখন ৮৮৫ কোনো ইন্সট্রাকশন (যেমন: দুটি সংখ্যা যোগ করা বা মেমোরি থেকে ডাটা আনা) এক্সিকিউট বা রান করে, তখন কোন সিগন্যাল কখন হাই (\$1\$) হচ্ছে, কখন লো (\$0\$) হচ্ছে এবং পুরো কাজটা শেষ হতে ঠিক কতটুকু সময় লাগছে—সেটি এই ডায়াগ্রামের মাধ্যমে চিত্রের আকারে প্রকাশ করা হয়।

- graphical representation where the 8085 instruction timing diagram represents the execution time of each instruction in graphical format.

২. দরকারী কিছু টার্মস (Useful Terms)

- **Clock Signal:** The time required to execute an instruction is called a clock cycle.
- **Machine Cycle:** The time required to access memory or input/output devices is called a machine cycle. The 8085 has 5 basic machine cycles i.e., load opcode, read from memory, write to memory, read I/O, and write I/O.
- **T-State:** A machine cycle and an instruction cycle take several clock periods. The portion of an operation performed in one system clock period is called a T-state.
- **Control Signals:** The control signal controls the operations. Common signals are ALE (address block enable), RD (read), WR (write), and IO/M (input/output) memory.

টাইমিং ডায়াগ্রাম বুঝতে গেলে এই ৪টি জিনিস জানা খুব জরুরি:

- **Clock Signal (ক্লক সিগন্যাল):** প্রসেসরের হৃদস্পন্দনের মতো। এটি একটি নিয়মিত স্পন্দন (Pulse) যা প্রসেসরের সব কাজকে তাল মিলিয়ে চলতে সাহায্য করে।
- **T-State:** এটি সময়ের ক্ষুদ্রতম একক। ক্লক সিগন্যালের একটি কমপ্লিট সাইকেল (একটি ওঠা এবং একটি নামা)-কে বলা হয় ১টি T-state।
- **Machine Cycle (মেশিন সাইকেল):** প্রসেসর যখন বাইরের কোনো ডিভাইসের সাথে যোগাযোগ করে—যেমন মেমোরি থেকে কিছু পড়া (Read), মেমোরিতে কিছু লেখা (Write) বা কোনো ইনপুট/আউটপুট ডিভাইসের সাথে কাজ করা—তখন সেই পুরো কাজটা করতে প্রসেসরের যেটুকু সময় লাগে, তাকে বলে মেশিন সাইকেল। একটি মেশিন সাইকেল কয়েকটি T-state নিয়ে গঠিত হয়।
- **Control Signals (কন্ট্রোল সিগন্যাল):** প্রসেসর যখন কোনো কাজ করে, তখন সে কিছু নিয়ন্ত্রণকারী সিগন্যাল পাঠায়। যেমন:

- **ALE (Address Latch Enable):** এটি মেমোরি অ্যাড্রেস লক বা ফিক্সড করার জন্য ব্যবহৃত হয়। (নোট: আপনার স্লাইডে এটি ভুলবশত 'address block enable' লেখা আছে, আসল নাম **Address Latch Enable**)।
- **RD (Read) & WR (Write):** ডাটা পড়ার জন্য RD এবং ডাটা লেখার জন্য WR সিগন্যাল পাঠানো হয়।
- **IO/M:** প্রসেসর কি মেমোরির (\$M\$) সাথে কাজ করছে নাকি কোনো ইনপুট/আউটপুট (\$IO\$) ডিভাইসের সাথে, তা এই সিগন্যাল দিয়ে ঠিক করা হয়।

৩. Machine Cycle of 8085 (৮০৮৫-এর মেশিন

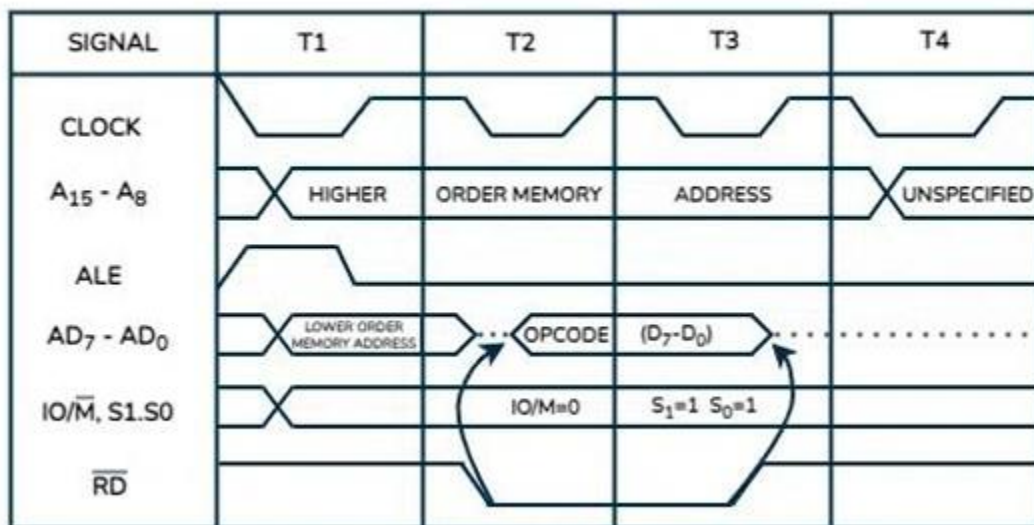
সাইকেলসমূহ)

৮০৮৫ মাইক্রোপ্রসেসরের মূলত ৫টি মৌলিক মেশিন সাইকেল থাকে। প্রসেসর যেকোনো কাজ করুক না কেন, সে এই ৫টি সাইকেলের কম্বিনেশন ব্যবহার করেই করে:

মেশিন সাইকেলের নাম	কাজ কী?	কত সময় লাগে? (T-states)
Opcode Fetch	মেমোরি থেকে মেইন কমান্ড বা ইন্সট্রাকশনটা প্রসেসরের ভেতরে নিয়ে আসা।	4T (সাধারণত ৪টি T-state লাগে)
Memory Read	মেমোরির নির্দিষ্ট কোনো ঠিকানা থেকে ডাটা প্রসেসরে পড়া।	3T
Memory Write	প্রসেসর থেকে কোনো ডাটা মেমোরির নির্দিষ্ট ঠিকানায় লিখে রাখা।	3T
I/O Read	বাইরের কোনো ইনপুট ডিভাইস (যেমন কিবোর্ড) থেকে ডাটা নেওয়া।	3T
I/O Write	বাইরের কোনো আউটপুট ডিভাইসে (যেমন ডিসপ্লে) ডাটা পাঠানো।	3T

১. Opcode Fetch Machine Cycle (অপকোড ফেচ সাইকেল)

সহজ ভাষায়: প্রসেসর যখন কোনো কাজ শুরু করে, তখন মেমোরি থেকে মেইন কমান্ড বা ইনস্ট্রাকশনটা (যেমন: MOV, ADD) নিজের ভেতরে টেনে নিয়ে আসাকে **Opcode Fetch** বলে। এটি করতে প্রসেসরের **4T-states** (T1, T2, T3, T4) সময় লাগে।



ডায়াগ্রামে টাইমিং সিগন্যালগুলো ধাপে ধাপে কাজ করে:

- **T1 স্টেট:**
 - ঠিকানা খোঁজা: প্রসেসর মেমোরির কোন জায়গা থেকে অপকোড আনবে, তার হাই-অর্ডার অ্যাড্রেস (A₁₅ - A₈) এবং লো-অর্ডার অ্যাড্রেস (AD₇ - AD₀) লাইনে পাঠিয়ে দেয়।
 - ALE = High: এই সময়ে ALE সিগন্যালটি হাই (1) হয়। এর মানে হলো, এটি নিচের বাসটিকে (AD₇ - AD₀) নির্দেশ দেয় যে, "এখন তোমার ভেতর দিয়ে অ্যাড্রেস যাচ্ছে, ডাটা নয়!"
 - স্ট্যাটাস সিগন্যাল: যেহেতু এটি মেমোরির কাজ, তাই IO/ $\overline{M}$ = 0 (অর্থাৎ মেমোরি অ্যাক্টিভ) এবং অপকোড ফেচের জন্য S₁ = 1, S₀ = 1 ফিক্সড হয়ে যায়।
- **T2 ও T3 স্টেট:**
 - ALE = Low: ALE এখন লো (0) হয়ে যায়, যার ফলে AD₇ - AD₀ লাইনটি এখন অ্যাড্রেস ছেড়ে ডাটা/অপকোড নেওয়ার জন্য তৈরি হয়।
 - $\overline{RD}$ = Low: রিড সিগন্যাল লো হওয়া মানে প্রসেসর মেমোরিকে বলছে, "এবার ডাটা পাঠাও"। মেমোরি তখন ওই নির্দিষ্ট অপকোডটি AD₇ - AD₀ লাইনে ছেড়ে দেয় এবং প্রসেসর সেটি রিড করে।

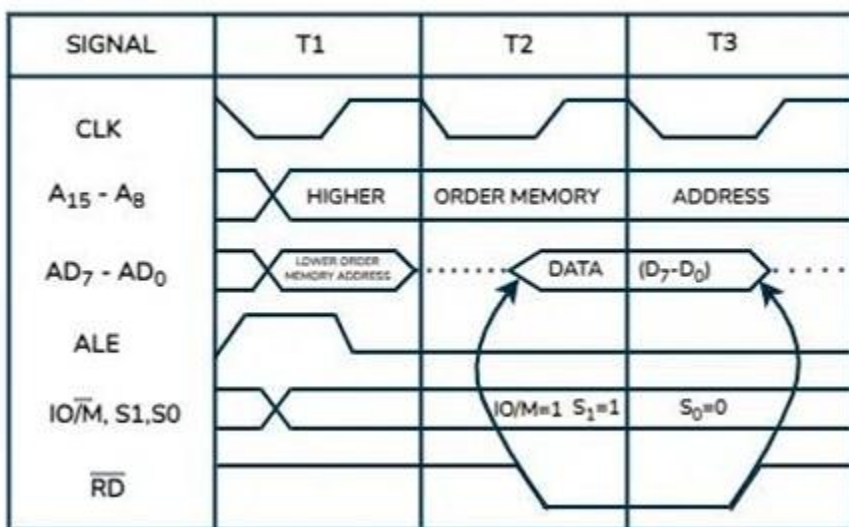
T4 স্টেট:

T4 স্টেট:

- ইন্টারনাল ডিকোডিং: প্রথম ৩ স্টেটে অপকোডটি প্রসেসরের ভেতরে চলে আসে। এই ৪র্থ স্টেটে প্রসেসর নিজে নিজে ঠাণ্ডা মাথায় চিন্তা করে বা ডিকোড করে যে, "আচ্ছা, কমান্ডটার মানে কী এবং এখন আমাকে কী করতে হবে?"।

২. Memory Read Machine Cycle (মেমোরি রিড সাইকেল)

সহজ ভাষায়: অপকোড তো চলে এলো। এবার ওই ইনস্ট্রাকশনটা পূরণ করার জন্য যদি মেমোরি থেকে সাধারণ কোনো ডাটা বা অপারেন্ড (যেমন: কোনো সংখ্যা) প্রসেসরে নিয়ে আসতে হয়, তাকে **Memory Read** বলে। এটি করতে প্রসেসরের **3T-states** (T1, T2, T3) সময় লাগে।



এটি দেখতে অনেকটা Opcode Fetch-এর মতোই, কিন্তু কিছু সূক্ষ্ম ও গুরুত্বপূর্ণ পার্থক্য আছে:

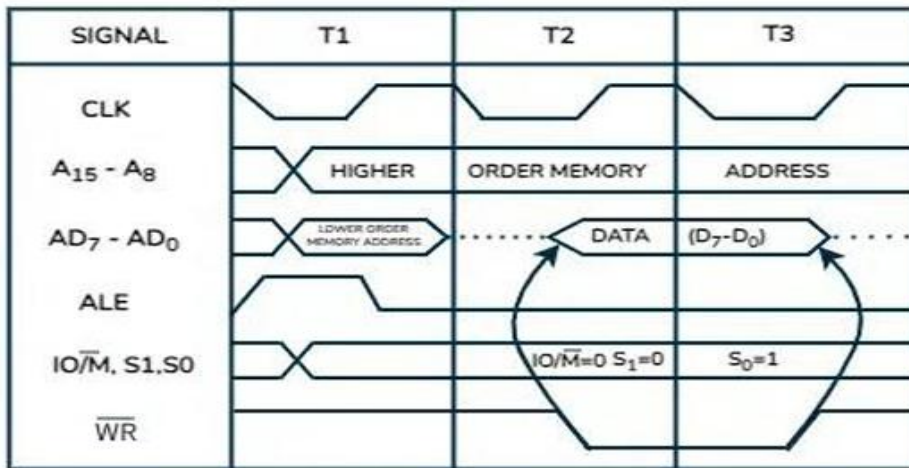
এটি দেখতে অনেকটা Opcode Fetch-এর মতোই, কিন্তু কিছু সূক্ষ্ম ও গুরুত্বপূর্ণ পার্থক্য আছে:

- সময় কম লাগে: এখানে কোনো ডিকোডিংয়ের ঝামেলা নেই, শুধু ডাটা এনেই কাজ শেষ। তাই কোনে T4 স্টেট লাগবে না, মাত্র 3T স্টেটেই কাজ খালাস।
- সিগন্যালের পার্থক্য:
 - এটিও মেমোরির কাজ, তাই এখানেও $IO/\overline{M} = 0$ থাকবে।
 - তবে স্ট্যাটাস সিগন্যাল বদলে যাবে। মেমোরি রিডের জন্য $S_1 = 1$ এবং $S_0 = 0$ হয়। (নোট: আপনার স্লাইডের ডায়াগ্রামে প্রিন্টিং মিস্টেক আছে, ওখানে ভুল করে $IO/\overline{M} = 1$ লেখা আছে, কিন্তু মেমোরি রিডের ক্ষেত্রে অবশ্যই $IO/\overline{M} = 0$ হবে, যা স্ট্যান্ডার্ড নিয়ম।)
- বাকি সব এক: T1-এ ALE হাই হয়ে অ্যাড্রেস লক করবে, T2 ও T3-তে $\overline{RD}$ লো হয়ে মেমোরি থেকে ডাটা প্রসেসরের ভেতরে নিয়ে আসবে।

Memory Write Machine Cycle (মেমোরি রাইট সাইকেল)

সহজ ভাষায়: প্রসেসর যখন নিজের ভেতরের কোনো ডাটা বা হিসাব-নিকাশের ফলাফল মেমোরির কোনো নির্দিষ্ট ঠিকানায় গিয়ে **জমা রাখতে বা লিখে রাখতে (Write)** চায়, তখন এই সাইকেলটি ব্যবহার করা হয়। এটি সম্পন্ন করতেও প্রসেসরের মোট **3T-states** (T1, T2, T3) সময় লাগে।

ডায়াগ্রাম দেখে স্টেপগুলো বুঝে নাও:



○ T1 স্টেট (ঠিকানা নির্ধারণ):

- প্রসেসর মেমোরির ঠিক কোন জায়গায় ডাটা রাখবে, তার ১৬-বিটের অ্যাড্রেসটি বাসে পাঠিয়ে দেয়। হাই-অর্ডার অ্যাড্রেস যায় $A_{15} - A_8$ লাইনে, আর লো-অর্ডার অ্যাড্রেস যায় $AD_7 - AD_0$ লাইনে।
- **ALE = High:** ALE সিগন্যালটি হাই (1) হয়ে লো-অর্ডার অ্যাড্রেসটিকে মেমোরিতে লক বা ফিক্সড করে দেয়।
- **কন্ট্রোল স্ট্যাটাস:** যেহেতু এটিও মেমোরির কাজ, তাই $IO/\overline{M} = 0$ (মেমোরি সিলেক্টেড)। কিন্তু এটি রাইট (Write) অপারেশন হওয়ায় স্ট্যাটাস পিনগুলোর মান বদলে যাবে: $S_1 = 0$ এবং $S_0 = 1$ ।

○ T2 ও T3 স্টেট (ডাটা পাঠানো ও রাইট করা):

- **$\overline{WR} = \text{Low}$:** এই সাইকেলের সবচেয়ে মেইন সিগন্যাল হলো এটি। T2 স্টেটের শুরুতেই প্রসেসর $\overline{WR}$ (Write) সিগন্যালটিকে লো (0) করে দেয়। এর মানে প্রসেসর মেমোরিকে সিগন্যাল দিচ্ছে, "আমি ডাটা পাঠাচ্ছি, জলদি লিখে নাও!"
- **ডাটা প্লেসমেন্ট:** $AD_7 - AD_0$ লাইনটি থেকে অ্যাড্রেস সরে যায় এবং প্রসেসর তার নিজের ভেতরের ডাটাটি (Data $D_7 - D_0$) এই লাইনে বসিয়ে দেয়।
- T3 স্টেটের শেষের দিকে যখন $\overline{WR}$ সিগন্যালটি আবার হাই (1) হয়ে যায়, ঠিক সেই মুহুর্তে মেমোরি ডাটাটি পাকাপোক্তভাবে নিজের ভেতরে সেভ বা রাইট করে নেয়।

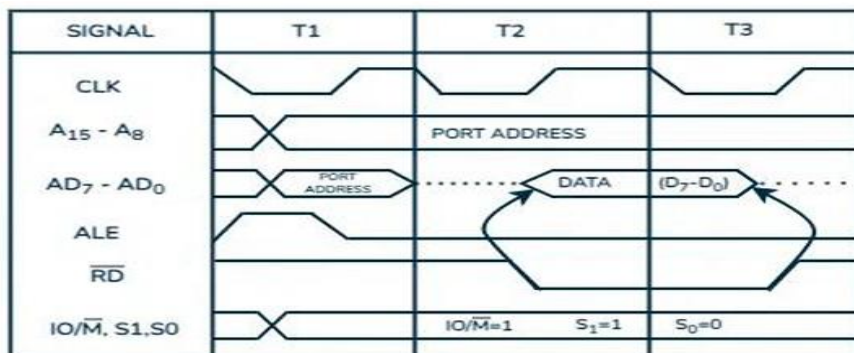
Memory Read বনাম Memory Write (ঝটপট পার্থক্য)

বৈশিষ্ট্য	Memory Read Cycle	Memory Write Cycle
প্রধান সিগন্যাল	$\overline{RD}$ পিনটি লো (0) হয়।	$\overline{WR}$ পিনটি লো (0) হয়।
স্ট্যাটাস সিগন্যাল	$S_1 = 1, S_0 = 0$	$S_1 = 0, S_0 = 1$
ডাটার দিক	মেমোরি থেকে ডাটা প্রসেসরে আসে।	প্রসেসর থেকে ডাটা মেমোরিতে যায়।

১. I/O Read Machine Cycle (আই/ও রিড সাইকেল)

সহজ ভাষায়: প্রসেসর যখন মেমোরি বাদ দিয়ে বাইরের কোনো ইনপুট ডিভাইস (যেমন: কিবোর্ড বা কোনো সেন্সর)-এর নির্দিষ্ট পোর্ট থেকে ১ বাইট ডাটা নিজের ভেতরে পড়তে বা নিয়ে আসতে চায়, তখন এই সাইকেলটি ব্যবহার করা হয়। এটি সম্পন্ন করতে প্রসেসরের 3T-states (T1, T2, T3) সময় লাগে।

ডায়াগ্রামের মূল ট্রিক ও সিগন্যাল:

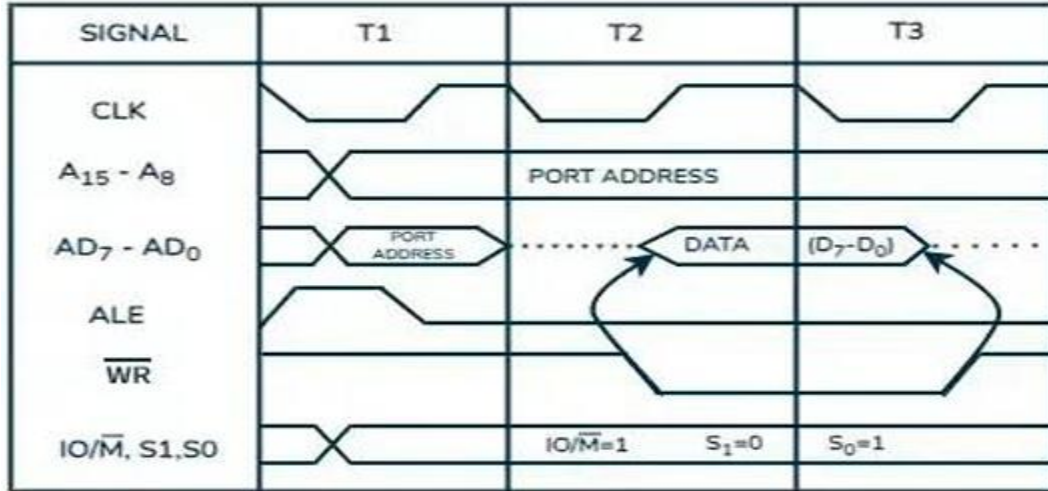


- $IO/\overline{M} = 1$ (সবচেয়ে গুরুত্বপূর্ণ পার্থক্য): যেহেতু এটি মেমোরির কাজ নয়, এটি ইনপুট/আউটপুট বা পোর্টের কাজ, তাই এবার $IO/\overline{M}$ পিনের মান হাই বা 1 হবে। (মনে আছে তো? মেমোরির ক্ষেত্রে এটি 0 ছিল)।
- ঠিকানা বা পোর্ট অ্যাড্রেস: T1 স্টেটে প্রসেসর যে পোর্টের ডাটা পড়বে তার ৮-বিটের পোর্ট অ্যাড্রেসটি $A_{15} - A_8$ এবং $AD_7 - AD_0$ উভয় লাইনেই ডুপ্লিকেট করে পাঠিয়ে দেয়। ALE হাই হয়ে যথারীতি অ্যাড্রেস লক করে।
- ডাটা রিড: T2 ও T3 স্টেটে $\overline{RD}$ সিগন্যাল লো (0) হয় এবং প্রসেসর $AD_7 - AD_0$ বাসের মাধ্যমে বাইরের ডিভাইস থেকে ডাটা ভেতরে নিয়ে আসে।
- স্ট্যাটাস পিন: রিড অপারেশনের জন্য সবসময় $S_1 = 1, S_0 = 0$ থাকে।

২. I/O Write Machine Cycle (আই/ও রাইট সাইকেল)

সহজ ভাষায়: প্রসেসর যখন তার ভেতরের কোনো ডাটা বাইরের কোনো আউটপুট ডিভাইস (যেমন: এলইউ ডিসপ্লে বা কোনো মোটর কন্ট্রোলার)-এর নির্দিষ্ট পোর্টে পাঠাতে বা লিখতে চায়, তখন এই সাইকেলটি ব্যবহৃত হয়। এটি সম্পন্ন করতেও প্রসেসরের 3T-states (T1, T2, T3) সময় লাগে।

ডায়াগ্রামের মূল ট্রিক ও সিগন্যাল:

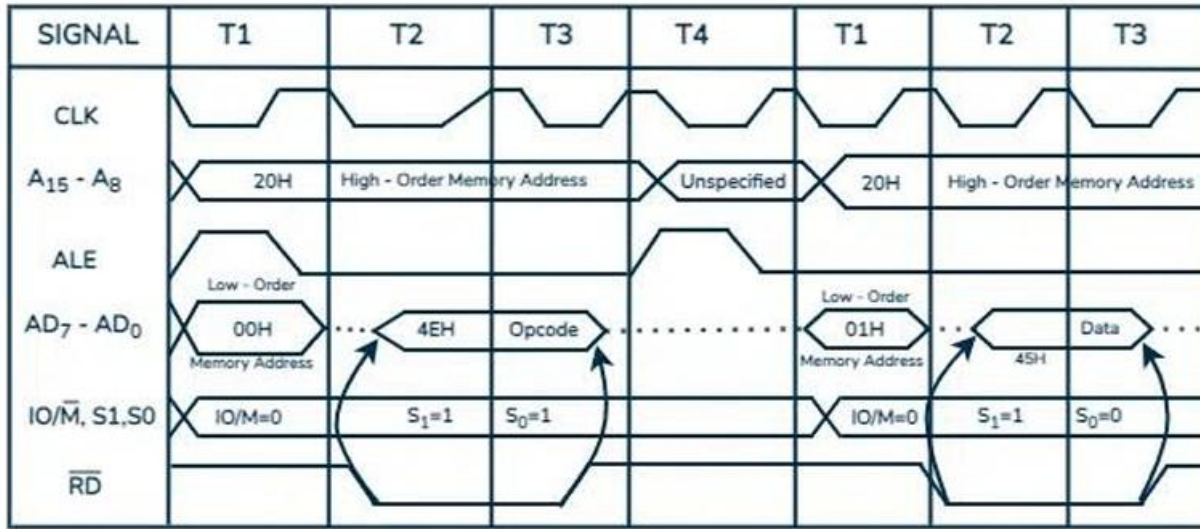


- $IO/\overline{M} = 1$: এটিও আই/ও অপারেশন, তাই এই সিগন্যালটি হাই বা 1 থাকবে।
- $\overline{WR} = 0$: যেহেতু আমরা বাইরে ডাটা পাঠাচ্ছি বা লিখছি, তাই এখানে $\overline{RD}$ পিন নিষ্ক্রিয় থাকবে এবং $\overline{WR}$ (Write) পিনটি T2 ও T3 স্টেটের জন্য লো (0) হয়ে যাবে। প্রসেসর তার ডাটা AD₇ - AD₀ বাসে ছেড়ে দেবে যাতে আউটপুট ডিভাইসটি তা গ্রহণ করতে পারে।
- স্ট্যাটাস পিন: রাইট অপারেশনের চিরন্তন নিয়ম অনুযায়ী এখানেও S₁ = 0, S₀ = 1 হবে।

মেমোরি (Memory) বনাম আই/ও (I/O) মনে রাখার মাস্টার ট্রিকস!

1. কাজের জায়গা ($IO/\overline{M}$ পিন):
 - মেমোরির যেকোনো কাজ (Memory Read/Write) হলে ডায়াগ্রামে $IO/\overline{M} = 0$ দেবে।
 - আই/ও-র যেকোনো কাজ (I/O Read/Write) হলে ডায়াগ্রামে $IO/\overline{M} = 1$ দেবে।
2. কাজের ধরন ($\overline{RD}$, $\overline{WR}$ এবং S₁, S₀):
 - রিড (Read) অপারেশন হলেই $\overline{RD}$ লো হবে, এবং S₁ = 1, S₀ = 0 হবে।
 - রাইট (Write) অপারেশন হলেই $\overline{WR}$ লো হবে, এবং S₁ = 0, S₀ = 1 হবে।

স্লাইডের ভাষায় সহজ বিশ্লেষণ (MVI A, 45H)



স্লাইডের ভাষায় সহজ বিশ্লেষণ (MVI A, 45H)

MVI A, 45H ইনস্ট্রাকশনটির মানে হলো: Accumulator (A)-এর ভেতরে সরাসরি 45H ডাটাটি মুভ বা লোড করো।

এই পুরো কাজটা করতে প্রসেসরের মেমোরিতে ২টি আলাদা জায়গায় যেতে হয় (যেমনটা টেবিলে দেখছ):

- 2000H ঠিকানায়: সেখানে আসল কমান্ডের কোড বা অপকোড 4EH লুকিয়ে আছে।
- 2001H ঠিকানায়: সেখানে আমাদের কাম্বিন্ট ডাটা 45H বসে আছে।

যেহেতু প্রসেসরকে মেমোরি থেকে ২টি আলাদা জিনিস এনে কাজটা শেষ করতে হয়, তাই এই ইনস্ট্রাকশনটি চালাতে ২টি মেশিন সাইকেল লাগে:

১. Opcode Fetch Machine Cycle (M1 সাইকেল - 4T)

- উদ্দেশ্য: 2000H মেমোরি লোকেশন থেকে মেইন কমান্ড বা অপকোড (4EH) প্রসেসরের ভেতরে আনা।
- ডায়াগ্রামের সিগন্যাল: * T1: হাই-অর্ডার বাসে যাবে 20H আর লো-অর্ডার বাসে যাবে 00H (মিলে হলো 2000H অ্যাড্রেস)। ALE হাই হয়ে এটি লক করবে।
 - T2 ও T3: $\overline{RD}$ লো হবে এবং মেমোরি থেকে 4EH অপকোডটি প্রসেসরে আসবে।
 - T4: প্রসেসর নিজে চিন্তা করবে, "আচ্ছা, ডিকোড করে বুঝলাম এটা MVI কমান্ড, মানে এর ঠিক পরের মেমোরিতে একটা ডাটা আছে যা আমাকে রিড করতে হবে।"

২. Memory Read Machine Cycle (M2 সাইকেল - 3T)

- উদ্দেশ্য: ঠিক তার পরের অ্যাড্রেস 2001H থেকে আসল ডাটা (45H) প্রসেসরে পড়া।
- ডায়াগ্রামের সিগন্যাল:
 - T1: এবার অ্যাড্রেস এক বেড়ে যাবে। হাই বাসে 20H এবং লো বাসে 01H (2001H অ্যাড্রেস)। ALE আবার হাই হবে।
 - T2 ও T3: $\overline{RD}$ আবার লো হবে এবং ওই ঠিকানা থেকে আসল ডাটা 45H প্রসেসরের Accumulator (A)-এ চলে আসবে।

টোটাল সময়: এই একটা ইনস্ট্রাকশন শেষ করতে প্রসেসরের মোট $4T + 3T = 7T$ স্টেট সময় লাগে।

স্টোরেজের ভাষায় সহজ বিশ্লেষণ (STA 8000H)

STA 8000H (Store Accumulator)-এর মানে হলো: Accumulator (A) রেজিস্টারের ভেতরে এই মুহুর্তে যে ডাটাটি আছে, তাকে মেমোরির 8000H নামক ঠিকানায় গিয়ে পার্মানেন্টলি জমা বা সেভ করে রেখে আসো।

এই পুরো কাজটা করতে প্রসেসরের মেমোরিতে ৪ বার আলাদা আলাদা অ্যাক্সেস করতে হয় (৪টি মেশিন সাইকেল লাগে):

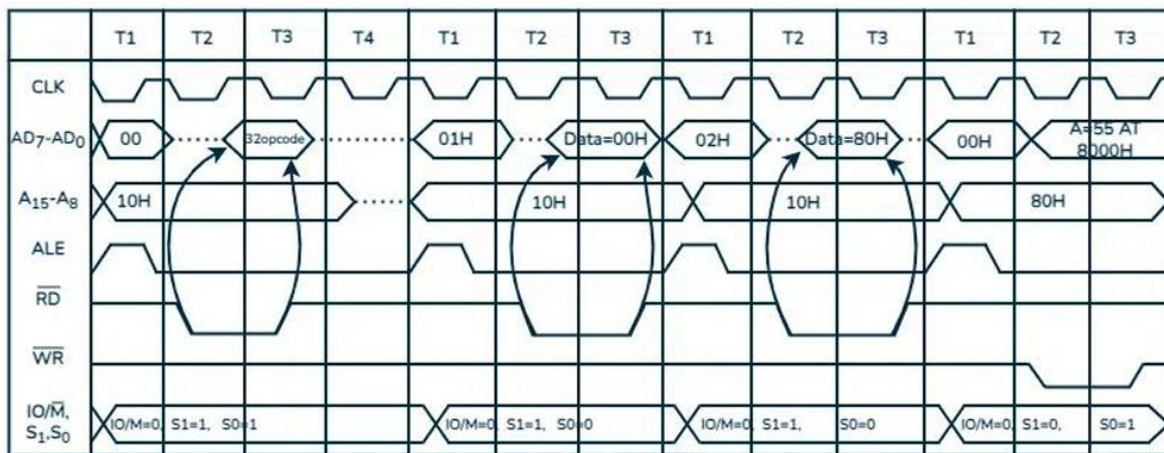
- 1000H লোকেশনে: মেইন কমান্ডের অপকোড 32H থাকে।
- 1001H লোকেশনে: টার্গেট অ্যাড্রেসের লোয়ার-বাইট 00H থাকে।
- 1002H লোকেশনে: টার্গেট অ্যাড্রেসের হায়ার-বাইট 80H থাকে।
- 8000H লোকেশনে (আসল কাজ): অবশেষে এই ঠিকানায় গিয়ে Accumulator-এর ডাটাটি রাইট (Write) করা হয়।

চলুন ধাপে ধাপে ৪টি মেশিন সাইকেল ডায়াগ্রামের সিগন্যালসহ দেখে নিই:

M1: Opcode Fetch (4T-states)

- উদ্দেশ্য: 1000H মেমোরি থেকে মেইন কমান্ডের অপকোড (32H) প্রসেসরে আনা।
- সিগন্যাল: T1-এ হায়ার বাস $A_{15} - A_8$ -এ যাবে 10H এবং লোয়ার বাসে যাবে 00H। ALE হাই হয়ে অ্যাড্রেস লক করবে। T2 ও T3-তে $\overline{RD}$ লো হয়ে অপকোড 32H প্রসেসরে ঢুকবে। T4-এ প্রসেসর ডিকোড করে বুঝবে, "এটি STA কমান্ড, এরপরে ২ বাইটের একটি মেমোরি অ্যাড্রেস আছে।" ($S_1 = 1, S_0 = 1$)

Address	Mnemonics	Opcode
1000H	STA 8000H	32H
1001H		00H
1002H		80H



M2: Memory Read (3T-states)

- উদ্দেশ্য: তার পরের মেমোরি 1001H থেকে অ্যাড্রেসের লোয়ার পার্ট (00H) রিড করা।
- সিগন্যাল: হায়ার বাসে 10H এবং লোয়ার বাসে 01H যাবে। $\overline{RD}$ লো হয়ে 00H ডাটাটি প্রসেসরের ভেতরের সাময়িক একটা জায়গায় জমা হবে। ($S_1 = 1, S_0 = 0$)

M3: Memory Read (3T-states)

- উদ্দেশ্য: তার পরেরিল মেমোরি 1002H থেকে অ্যাড্রেসের হায়ার পার্ট (80H) রিড করা।
- সিগন্যাল: হায়ার বাসে 10H এবং লোয়ার বাসে 02H যাবে। $\overline{RD}$ লো হয়ে 80H ডাটাটি ভেতরে আসবে। এখন প্রসেসরের কাছে পুরো ট্যাগেট অ্যাড্রেসটি রেডি হলো: 8000H। ($S_1 = 1, S_0 = 0$)

M4: Memory Write (3T-states - আসল অ্যাকশন!)

- উদ্দেশ্য: এইমাত্র খুঁজে পাওয়া 8000H অ্যাড্রেসে গিয়ে Accumulator-এর ভেতরের ডাটাটি (ধরো C7H বা স্লাইডের ডায়াগ্রাম অনুযায়ী 55H) লিখে ফেলা।
- সিগন্যাল: এবার হায়ার বাসে যাবে নতুন অ্যাড্রেস 80H এবং লোয়ার বাসে যাবে 00H। যেহেতু এটি Write অপারেশন, তাই এবার $\overline{RD}$ চুপ থাকবে, কিন্তু $\overline{WR}$ সিগন্যালটি লো (0) হবে। প্রসেসর তার নিজের Accumulator-এর ডাটাটি বাসে ছেড়ে দেবে এবং মেমোরি তা সেভ করে নেবে। ($S_1 = 0, S_0 = 1$)

টোটাল সময়: $4T + 3T + 3T + 3T = 13T$ স্টেট।

স্লাইডের ভাষায় সহজ বিশ্লেষণ ($IN\ 80H$ এবং $OUT\ 80H$)

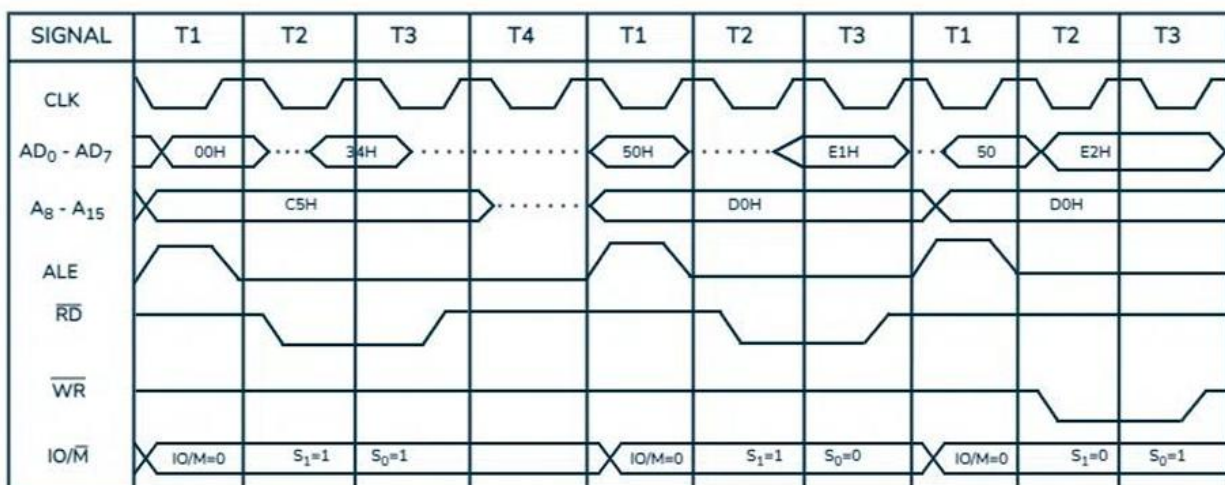
এখানে দুটি ভিন্ন ইনস্ট্রাকশন এবং তাদের টাইমিং ডায়াগ্রাম দেখানো হয়েছে। দুটির স্ট্রাকচার ছব্ব এক, শুধু ডাটার দিক আলাদা।

১. $IN\ 80H$ ইনস্ট্রাকশন (ইনপুট ডিভাইস থেকে ডাটা নেওয়া)

এর মানে হলো: $80H$ নম্বরের ইনপুট পোর্ট (যেমন কিবোর্ড) থেকে ১ বাইট ডাটা টেনে প্রসেসরের Accumulator (A)-এ নিয়ে আসো।

এটি সম্পন্ন করতে প্রসেসরের ৩টি মেশিন সাইকেল ($M1, M2, M3$) এবং মোট 10T-states ($4T + 3T + 3T$) সময় লাগে:

- **M1: Opcode Fetch (4T):** মেমোরি থেকে IN কমান্ডের অপকোড (DBH) প্রসেসরে নিয়ে আসা এবং ডিকোড করা।
- **M2: Memory Read (3T):** মেমোরির ঠিক পরের ঘর থেকে পোর্টের ঠিকানা $80H$ রিড করে প্রসেসরের ভেতরে আনা।
- **M3: I/O Read (3T):** এই ধাপে আসল কাজ হয়। প্রসেসর $80H$ পোর্ট অ্যাড্রেসটি বাসে পাঠায়। যেহেতু এটি আই/ও অপারেশন, তাই $IO/\overline{M} = 1$ এবং $\overline{RD} = 0$ হয়। বাইরের ইনপুট ডিভাইস তখন ডাটা বাসে ছেড়ে দেয় এবং ডাটা প্রসেসরের Accumulator-এ চলে আসে।



২. OUT 80H ইনস্ট্রাকশন (আউটপুট ডিভাইসে ডাটা পাঠানো)

এর মানে হলো: Accumulator (A)-এ এই মুহূর্তে যে ডাটা আছে, তাকে 80H নম্বরের আউটপুট পোর্টে (যেমন ডিসপ্লে) পাঠিয়ে দাও।

এটি করতেও ৩টি মেশিন সাইকেল (M1, M2, M3) এবং মোট 10T-states ($4T + 3T + 3T$) সময় লাগে:

- **M1: Opcode Fetch (4T):** মেমোরি থেকে OUT কমান্ডের অপকোড (D3H) প্রসেসরে নিয়ে আসা।
- **M2: Memory Read (3T):** মেমোরির ঠিক পরের ঘর থেকে পোর্টের ঠিকানা 80H রিড করা।
- **M3: I/O Write (3T - আসল কাজ):** প্রসেসর এবার 80H পোর্টে ডাটা পাঠাবে। তাই এবারও $IO/\overline{M} = 1$ থাকবে, কিন্তু রাইট অপারেশন হওয়ায় $\overline{WR} = 0$ হবে। প্রসেসর তার Accumulator-এর ডাটা বাসে ছেড়ে দেয় এবং আউটপুট ডিভাইস তা রিসিভ করে।

বিশেষ নোট (পোর্ট ডুপ্লিকেশন): M3 সাইকেলে যখন ৮-বিটের পোর্ট অ্যাড্রেস (80H) পাঠানো হয়, তখন সেটি হাই-অর্ডার বাসেও ($A_{15} - A_8$) যায়, আবার লো-অর্ডার বাসেও ($AD_7 - AD_0$) যায়। অর্থাৎ দুই বাসেই একই ঠিকানা (80H এবং 80H) ডুপ্লিকেট হয়ে দেখায়।

(Applications of 8085 Timing Diagram)

- **সিস্টেমের ট্র্যাক রাখা (Track Changes):** টাইমিং ডায়াগ্রাম হলো এক ধরনের সিকোয়েন্স ডায়াগ্রাম। প্রসেসরের ভেতরে প্রতি মুহূর্তে (যেমন: প্রতি টি-স্টেটে) অ্যাড্রেস বাস, ডাটা বাস বা কন্ট্রোল সিগন্যালে কী কী পরিবর্তন হচ্ছে, তার নিখুঁত ট্র্যাক বা হিসাব এই ডায়াগ্রাম থেকে পাওয়া যায়।
- **ক্লক ফ্রিকোয়েন্সি নির্ধারণ (Planning Clock Frequency):** একটি মাইক্রোপ্রসেসর কতটা স্পিডে বা কত ফ্রিকোয়েন্সিতে (যেমন: 3 MHz) কাজ করবে, তা প্ল্যান করার জন্য টাইমিং ডায়াগ্রাম অপরিহার্য।
- **মেমোরি সার্কিট ডিজাইন (Designing Memory Circuits):** মেমোরি থেকে ডাটা পড়ার সময় বা লেখার সময় সিগন্যাল কতক্ষণ স্থায়ী হতে হবে (যাকে টেকনিক্যাল ভাষায় Setup Time এবং Hold Time বলে), তা টাইমিং ডায়াগ্রাম দেখে নিখুঁতভাবে ডিজাইন করা হয়। নতুবা ডাটা লস বা সিস্টেম ক্র্যাশ হতে পারে।
- **ইঞ্জিনিয়ারদের জন্য ট্রাবলশুটিং (Troubleshooting & Optimization):** কোনো ডিভাইস বা মেমোরির সাথে প্রসেসরের কানেকশনে কোনো সমস্যা (Bug) হলে ইঞ্জিনিয়াররা টাইমিং ডায়াগ্রাম দেখে দ্রুত ডিসিশন নিতে পারেন এবং প্রসেসরের পারফরম্যান্স সর্বোচ্চ লেভেলে অপটিমাইজ করতে পারেন।
- **স্টুডেন্টদের জন্য ভিজুয়াল গাইড (Visual Learning):** একজন স্টুডেন্টের জন্য প্রসেসরের ভেতরের জটিল মেকানিজম শুধু থিওরি পড়ে বোঝা কঠিন। টাইমিং ডায়াগ্রাম পুরো কাজটাকে একটা ছবির মতো চোখের সামনে ফুটিয়ে তোলে, যা শেখার প্রক্রিয়াকে পানির মতো সহজ করে দেয়।

Applications Of 8085

- System charts keep track of all changes occurring in the system.
- Timelines provide a more accurate representation of physical activity.
- The timing table of 8085 instructions graphically represents the time required to execute each instruction.
- Timing Diagram is a special form of sequence diagram.
- Timing Diagram keep track of every change that occur in the system.
- The timing diagram is important for planning the clock frequency of the [microprocessor](#).
- The timing diagram helps design memory circuits that meet the required set and hold times during read and write operations.
- The timing diagram of the 8085 microprocessor looks like a brief overview of how the microprocessor works during operation. It helps engineers solve problems, create good memories and interactions between devices, make quick decisions, and do good, on-time work. For students, it simplifies the learning process by providing a visual representation of the inner workings of a microprocessor. Essentially, it serves as a guide for understanding, troubleshooting and optimizing the performance of the 8085 microprocessor.